



Prof. Dr. Christoph Pflaum
Benjamin Mann

Summer Term
2024

Simulation and Scientific Computing 2 Assignment 1

Organizational Details

1. **Your Tutors:** Board exercises will be held by Benjamin Mann, the computer exercises will be hosted by Sebastian Bönning.
2. **Programming exercises:** The programming language for the implementation of the programming exercises is C++.
3. **Reference system:** The reference system on which we will compile and run your submission, are the computers in the CIP pools.
4. **Assignment sheets:** The assignment sheets, supplementary material, and further information will be available via the StudOn course. Also the submission of the solutions has to be done in StudOn.
5. **Team work:** In order to encourage team work we require you to form teams of **three** people submitting their solution together. Students are responsible for building teams. Submissions of groups with less than three people will **not** be accepted and corrected!
6. **Credits:** Each assignment will be graded with five points maximum. In order to pass the exercise classes, you have to acquire at least 13 points out of 20. Some assignment sheets feature bonus tasks that will allow you to obtain bonus points.
7. **Plagiarism:** Each team must submit their original solution. If a team ignores this, their submission will be graded with zero points.
8. **Computer exercises:** These exercises will take place at the time and place given in Campo/StudOn. During this time, the tutors will be available to answer your questions and help you finding errors in your code. It is **not** the tutors job to do the work for you or provide you with a solution!

Elliptic PDE with Multigrid

1. Your task is to implement a geometric multigrid (GMG) solver which computes an approximate solution to a discretization of the following elliptic partial differential equation (PDE):

$$\begin{cases} -\Delta u = f & \text{in } \Omega \\ u = g & \text{on } \partial\Omega, \end{cases} \quad (1)$$

with $\Omega = (0, 1) \times (0, 1) \subset \mathbb{R}^2$. The Dirichlet data g and the right-hand side f are defined by:

$$f(x, y) = 2\pi^2 \cos(\pi x) \cos(\pi y), \quad (2)$$

$$g(x, y) = \cos(\pi x) \cos(\pi y). \quad (3)$$

The problem is discretized by finite differences with mesh size h such that the grid comprises $(2^\ell + 1) \times (2^\ell + 1)$ vertices (including boundaries). This discretization leads to a linear system of equations (LSE) of the form

$$\mathbf{A}\mathbf{u} = \mathbf{b} \quad (4)$$

where the discrete solution u_h is related to the solution vector $\mathbf{u}$ s.th. $u_h(x_i, y_j) = [\mathbf{u}]_k$. with an appropriate mapping from grid points to vector components $(x_i, y_j) \mapsto k$.

This linear system shall be solved iteratively by means of a GMG with V(2,1)-cycle using ℓ levels with $\mathbf{u} = \mathbf{0}$ as initial guess for the interior of the domain. For the grid transfer, use *full weighting restriction* and *bi-linear interpolation*. For smoothing, use Gauss-Seidel iterations.

In order to increase the efficiency of your calculations, think about an optimal implementation in terms of data layout and update strategy.

We recommend a modular implementation of the different parts of your program in order to be easily adapt to other problems (e.g. a suitable `Grid` class).

2. Your program `mgsolve` should be able to be executed with the following command:

```
./mgsolve lvl iter
```

where `lvl` is the number of levels and `iter` the number of iterations, i.e., V-cycles.

3. After each V-cycle, the program should output the *relative residual norm* r as well as the estimated *convergence factor* q on the screen, where

$$r = \frac{\|\mathbf{r}^{(i)}\|}{\|\mathbf{r}^{(0)}\|}, \quad q = \frac{\|\mathbf{r}^{(i)}\|}{\|\mathbf{r}^{(i-1)}\|},$$

and $\mathbf{r}^{(i)} = \mathbf{A}\mathbf{u}^{(i)} - \mathbf{b}$ is the residual after the i -th iteration.

4. Furthermore, after each iteration, the *discrete L_2 error*

$$\|\mathbf{e}^{(i)}\|_{\tilde{L}^2} = \sqrt{\frac{1}{n}} \|\mathbf{e}^{(i)}\|$$

should also be written to the screen. Here, $n = (2^\ell - 1)^2$ is the number of unknowns. The error can be approximated by $\mathbf{e} = \mathbf{u} - I^h u$ where u is the analytic solution to (1), which can be obtained by having a closer look at the boundary data g from (3)) and I^h is an operator that restricts a function to the grid points at the finest level.

5. Write the approximate solution u_h to a file named `solution.txt`. Choose a file format, which can be visualized by `gnuplot[1]` such that the x-direction of the domain corresponds to the first dimension of the `gnuplot` output. You may use the `gnuplot` script provided via StudOn. If you don't, include your visualization toolchain in your submission.
6. Run your program with $\ell = 3, \dots, 13$ and 10 iterations. For each mesh size, plot the discrete L^2 error over the iterations using a logarithmic scale on the y-axis. Also plot the number of iterations i_{conv} required to reach *error convergence*, over the number of unknowns n . For this second plot, use a logarithmic scale on the x-axis. What can be said about the relation between i_{conv} and n ? In contrast, consider the number of iterations i_ε required to reach a certain residual reduction, e.g., $\varepsilon = 10^{10}$. How is the relation between i_ε and n ?
7. Please hand in your solution to this assignment until Wednesday, **May 08, 2024** at 23:55! Make sure the following requirements are met:
 - (a) The program should be compilable with a Makefile that you also have to provide.
 - (b) The program should compile without errors or warnings with the following g++ compiler flags:


```
-O3 -Wall -Winline -Wshadow -std=c++17
```
 - (c) The program should be callable as specified above and output the required values.
 - (d) The solution should contain **well commented** source files, a fitting Makefile that satisfies all the conditions specified above and instructions how to use your program (e.g. in form of a short README file).
 - (e) Submit your solution on StudOn as a team submission and include all your team members. Make sure that all required files are included in your submission!

Credits

In this assignment the credits are awarded in the following way:

1. Up to **five** points are awarded if your program correctly performs the above tasks, fulfills all of the above requirements and the graphs/tables show the correct behavior. Submissions with compile errors will lead to **zero** points!
2. Bonus task: **One bonus** point is awarded if your program and output is extended as follows: Implement the functionality to replace the first iteration with an FMG scheme. Note that this requires to correctly initialize the right hand side on each grid level! After the FMG completes, the remaining iterations should be performed as by the main task.

Repeat the experiment above with this setup. Is the relation between i_{conv} and n changed?

You can either provide a separate implementation file for the bonus task or add a switch (e.g. via the command line) to the program of the main task. Explain in the README file how to enable the bonus task.

References

- [1] <http://www.gnuplot.info/docs/gnuplot.pdf>
- [2] <https://www.studon.fau.de/crs3739925.html>